

# Práctica 1

## Parte 1

En esta primera parte de la práctica vamos a configurar el entorno de desarrollo que utilizaremos a lo largo del curso, que se compone de:

**Visual Studio Code (VSCode):** un entorno de desarrollo altamente adaptable y eficiente, reconocido por su interfaz intuitiva y su capacidad para integrar extensiones. Con funciones como resaltado de sintaxis y completado automático, VSCode proporciona un espacio de trabajo robusto y fácil de usar.

**PlatformIO:** una herramienta de desarrollo diseñada para simplificar la programación de sistemas empotrados. Funciona como una extensión para diferentes IDEs, entre ellos VSCode, abstrayendo la complejidad de la configuración del toolchain y las tareas asociadas al desarrollo (compilación, carga, etc.) para diferentes placas ([boards](#)), arquitecturas ([platforms](#)), y entornos ([frameworks](#)).

*Cada entorno es aplicable a diferentes arquitecturas; cada arquitectura está presente en diferentes placas.*

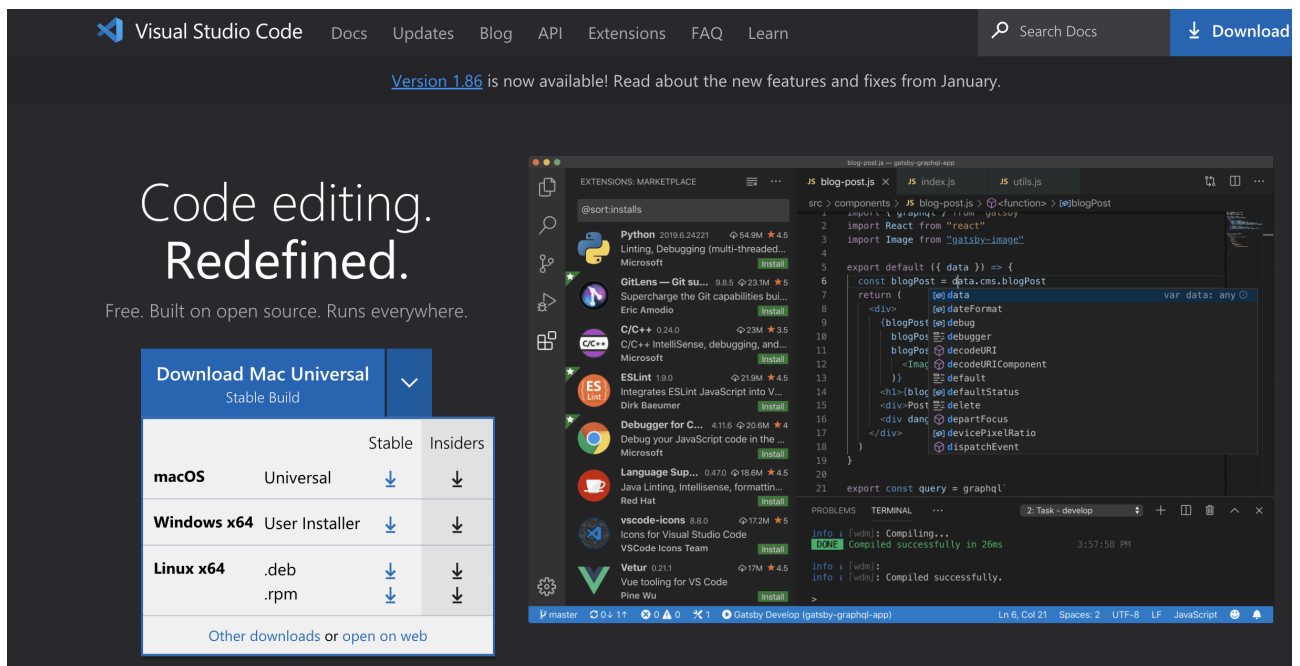
**ESP-IDF (Espressif IoT Development Framework):** es un entorno de desarrollo (un framework compatible en PlatformIO) dedicado para las arquitecturas de las placas del fabricante (esencialmente ESP32 y ESP8266 en sus diferentes versiones). Proporciona kernel, drivers, librerías y el toolchain (compilador, linker, bootloader, etc.) para programar sus placas, así como herramientas front-end para su fácil utilización.

**FreeRTOS:** un sistema operativo en tiempo real apropiativo multitarea de código abierto, conocido por su diseño modular. Ofrece programación multitarea, semáforos y servicios esenciales para el desarrollo de sistemas eficientes en tiempo real.

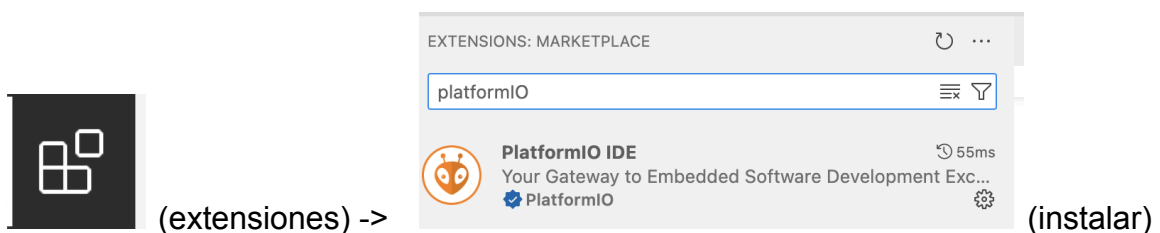
*ESP-IDF utiliza una versión adaptada de FreeRTOS como Kernel (Vanilla FreeRTOS), proporcionando además librerías propias de Espressif que, a modo de wrapper de FreeRTOS, proporcionan abstracción para sacar el máximo partido a su hardware. Para saber más: [FreeRTOS \(IDE\)](#).*

Para esta primera parte de la práctica tienes que escribir una memoria respondiendo a una serie de preguntas o aportando evidencia de algunas actividades. Estos apartados están indicados en rojo a lo largo del documento. Sigue los siguientes pasos:

1. Entra en <https://code.visualstudio.com>, descarga el IDE para tu sistema operativo e instálalo en tu sistema.

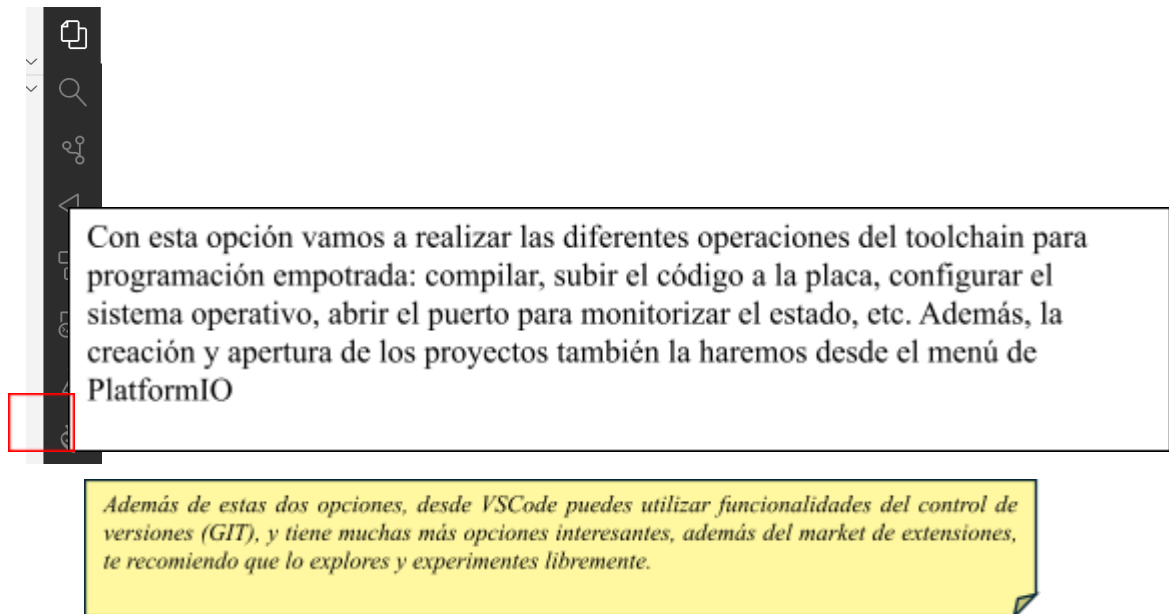


2. Abre VSCode e instala la extensión PlatformIO.



3. Una vez instalado, el entorno VSCode muestra una barra lateral como la siguiente:

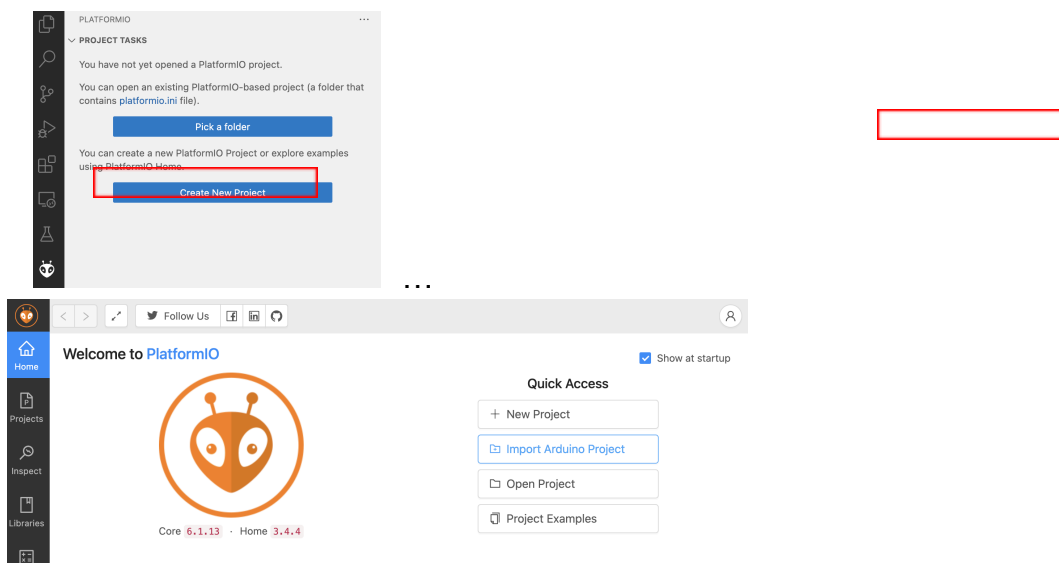
En esta opción tenemos una visión del sistema de archivos. La utilizaremos para abrir, crear, renombrar, mover, .. los diferentes archivos y carpetas de nuestro proyecto.



Con esta opción vamos a realizar las diferentes operaciones del toolchain para programación empotrada: compilar, subir el código a la placa, configurar el sistema operativo, abrir el puerto para monitorizar el estado, etc. Además, la creación y apertura de los proyectos también la haremos desde el menú de PlatformIO

*Además de estas dos opciones, desde VSCode puedes utilizar funcionalidades del control de versiones (GIT), y tiene muchas más opciones interesantes, además del market de extensiones, te recomiendo que lo explores y experimentes libremente.*

- Es hora de crear nuestro primer proyecto de prueba. Entra en las opciones de PlatformIO y selecciona crear nuevo proyecto. En la parte derecha aparecerá PlatformIO Home (si no estaba desde el principio), selecciona nuevo proyecto en ese menú.



The screenshot shows the PlatformIO Home interface. On the left, there's a sidebar with icons for Home, Projects, Inspect, and Libraries. The main area displays a welcome message and a 'Quick Access' section with buttons for '+ New Project', 'Import Arduino Project', 'Open Project', and 'Project Examples'. A red box highlights the 'Create New Project' button in the 'PROJECT TASKS' section.

- Rellena el formulario como aparece en la imagen.

Project Wizard

This wizard allows you to **create new** PlatformIO project or **update existing**. In the last case, you need to uncheck "Use default location" and specify path to existing project.

Name: STR2024\_P1\_1

Board: Espressif ESP32 Dev Module

Framework: Espidf

Location:

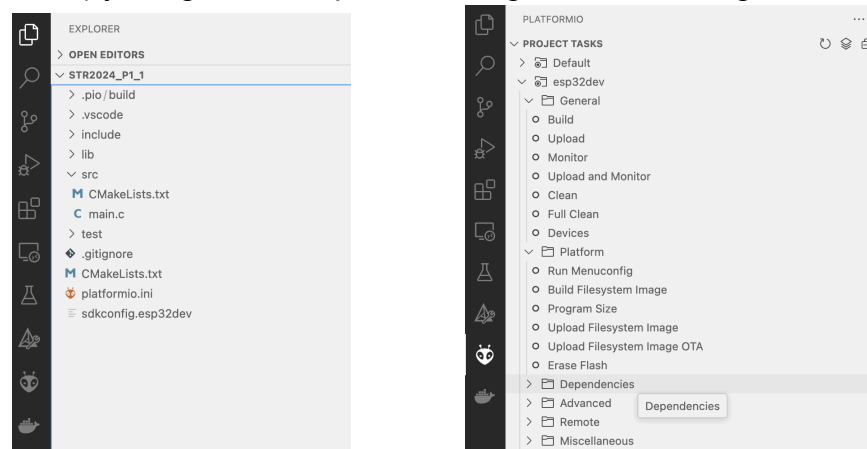
☒ Use default location

Cancel

Finish

*La ubicación por defecto suele ser <home>/Documents/PlatformIO/Projects. Puedes usar esta ubicación, o puedes crear una carpeta en tu espacio de usuario para la práctica. En cualquier caso, es importante que conozcas bien dónde se encuentra tu código.*

6. Cuando finalice, selecciona las dos vistas: sistema de archivos (izquierda) y PlatformIO (derecha) y asegúrate de que tienes algo similar a lo siguiente.



Un proyecto ESP-IDF en PlatformIO para VSCode sigue una estructura organizativa como la siguiente:

- **.pio y .vscode:** Carpetas ocultas que contienen configuraciones internas de PlatformIO y VSCode, respectivamente. Puedes ignorar su contenido en la mayoría de los casos.
- **build:** Carpeta que almacena los archivos generados durante el proceso de compilación (esta carpeta aparecerá en el momento de construir el binario). Aquí encontrarás el firmware compilado, así como archivos temporales y de construcción.

- **include y src:** Carpetas que contienen los archivos de encabezado (.h) y de código fuente (.c o .cpp), respectivamente, para tu proyecto principal. Aquí es donde escribirás la lógica principal de tu aplicación.
- **lib:** Carpeta opcional que puede contener bibliotecas de terceros o bibliotecas personalizadas que deseas incluir en tu proyecto.
- **test:** Carpeta opcional que puede contener pruebas unitarias para tu código.
- **CMakeLists.txt:** Archivo de configuración de CMake utilizado para describir cómo se debe construir tu proyecto.
- **platformio.ini:** Archivo de configuración de PlatformIO. Aquí especificas las configuraciones del proyecto, como la placa objetivo, la velocidad de baudios, y otras opciones relacionadas con la compilación y carga del firmware.

*Tómate tu tiempo para explorar estas carpetas y los archivos que contienen. Observa que, de momento, main.c solo contiene una función vacía definida como `void app_main()`. Este es el punto de entrada de los proyectos con ESP-IDF, y a partir de ella construiremos nuestro código.*

7. Vamos a introducir algo de código en el proyecto. Abre el fichero main.c (desde la vista de sistema de archivos) e introduce el siguiente código, cambiando <nombre> y <apellidos> por tu nombre y tus apellidos, respectivamente:

```
#include <stdio.h>
#include "esp_log.h"
#include "esp_chip_info.h"
#include "esp_clk_tree.h"

const char* TAG = "P1_1";

// Función para obtener el nombre del modelo como cadena
const char* get_chip_model_name(esp_chip_model_t model) {
    switch (model) {
        case CHIP_ESP32:      return "ESP32";
        case CHIP_ESP32S2:    return "ESP32-S2";
        case CHIP_ESP32S3:    return "ESP32-S3";
        case CHIP_ESP32C3:    return "ESP32-C3";
        case CHIP_ESP32C2:    return "ESP32-C2";
        case CHIP_ESP32C6:    return "ESP32-C6";
        case CHIP_ESP32H2:    return "ESP32-H2";
        case CHIP_POSIX_LINUX: return "POSIX/Linux Simulator";
        default:              return "Unknown";
    }
}

void app_main(void)
{
    // Estructura para almacenar información del chip
    esp_chip_info_t chip_info;
    // Entero para almacenar la frecuencia de la CPU
    uint32_t freq;

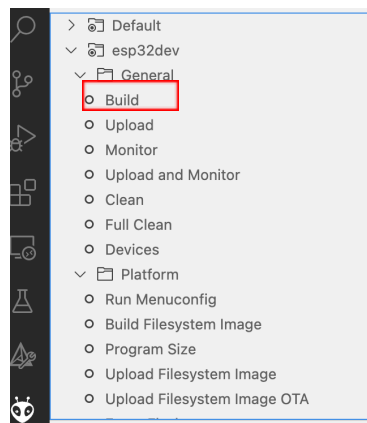
    ESP_LOGI(TAG, "Hello ESP32!");
    ESP_LOGI(TAG, "<Nombre1>, <Apellidos1>");
    ESP_LOGI(TAG, "<Nombre2>, <Apellidos2>");
    // ...
    // ESP_LOGI(TAG, "<NombreN>, <ApellidosN>");
}
```

```
// Obtener información del chip
esp_chip_info(&chip_info);
if (esp_clk_tree_src_get_freq_hz(SOC_MOD_CLK_CPU,
    ESP_CLK_TREE_SRC_FREQ_PRECISION_EXACT, &freq) != ESP_OK)
{
    ESP_LOGI(TAG, "Error obteniendo la frecuencia de la CPU");
    freq = 0;
}

// Formatear la cadena con el modelo de chip y el número de cores
ESP_LOGI(TAG, "Modelo: %s, Cores: %d, Frecuencia: %ld", get_chip_model_name(chip_info.model),
    chip_info.cores, freq);
}
```

*El entorno integrado de VSCode te permite navegar por el código hacia sus definiciones. Pulsa la letra **CONTROL** y clic en "esp\_chip\_info.h" para navegar al fichero de cabecera correspondiente. También puedes navegar con grano más fino haciendo lo propio con el nombre de las estructuras, prueba a seleccionar "esp\_chip\_info\_t".*

8. Para compilar el proyecto, desde el menú de PlatformIO, selecciona "build".



*El proceso de compilación con PlatformIO abrirá una terminal de salida en la parte inferior derecha del entorno. Si todo va correctamente, finalmente obtendrás un binario (.elf) que la arquitectura del chip puede ejecutar, solo falta escribirlo en la flash del chip.*

*Al inicio del log de compilación, PlatformIO te da información específica acerca del hardware y del entorno, como las versiones que está utilizando de las diferentes dependencias.*

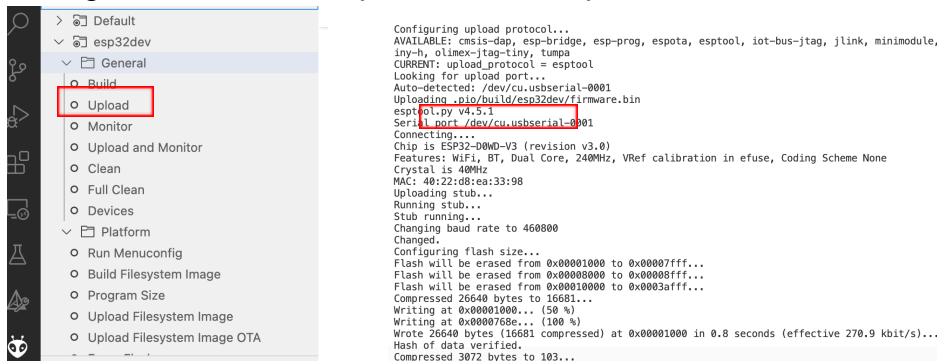
*Al final del log te da información del firmware que se ha generado así como del porcentaje de uso de la memoria que supone.*

#### **Apartado I: Primera compilación.**

*Responde a las siguientes preguntas y respáldalas con capturas de pantalla:*

- 1.- ¿Cuánta capacidad tiene la flash del hardware que estás utilizando?
- 2.- ¿Qué versión de espidf tiene el entorno que ha creado PlatformIO?
- 3.- ¿Cuánta memoria Flash ocupa el firmware generado? ¿Qué porcentaje supone del disponible?
- 4.- ¿Cuánta memoria RAM supone la ejecución de este código? ¿Qué porcentaje supone del disponible?
- 5.- ¿En qué ruta está el fichero .elf generado?

9. Para grabar este firmware en la placa, basta con conectarla con USB a nuestro ordenador y seleccionar la opción “upload” de PlatformIO. El propio entorno se encargará de detectar el puerto utilizado por ti.



*Si esta operación te da error, es posible que no tengas instalados los drivers del convertidor UART-USB que utilizan las placas ESP32. Descárgalo e instálalo para tu sistema operativo desde aquí:*

<https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers?tab=downloads>



10. Vamos a configurar el entorno para que utilice siempre el mismo puerto, esto evitará problemas en el caso de que conectes otros dispositivos a la vez. También vamos a definir la velocidad de transferencia de bytes a través de la UART (baudios). Abre el fichero platformio.ini y añade la siguiente línea:

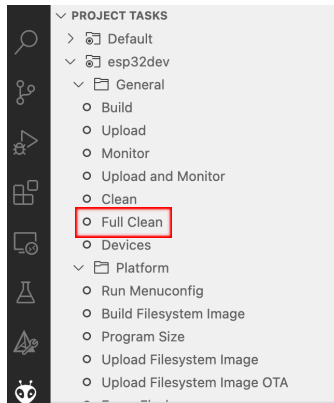
```
upload_port = <puerto detectado>
monitor_port = <puerto detectado>
monitor_speed = 115200
```

*En el ejemplo anterior sería:*

```
upload_port = /dev/cu.usbserial-0001
monitor_port = /dev/cu.usbserial-0001
monitor_speed = 115200
```

11. Para monitorizar la actividad por el Puerto serie, utiliza la opción monitor desde el menu de PlatformIO.





```

-|0:32m| (53) boot: Enabling RNG early entropy source...-|0m
-|0:32m| (58) boot: Partition Table:-|0m
-|0:32m| (62) boot: # Label Usage Type ST Offset Length-|0m
-|0:32m| (69) boot: 0 nvs WiFi data 01 02 00009000 00006000-|0m
-|0:32m| (77) boot: 1 phy_init RF data 01 01 0000f000 00001000-|0m
-|0:32m| (84) boot: 2 factory factory app 00 00 00010000 00100000-|0m
-|0:32m| (92) boot: End of partition table-|0m
-|0:32m| (96) esp_image: segment 0: paddr=00010020 vaddr=3f400020 size=09340h ( 37696) map-|0m
-|0:32m| (118) esp_image: segment 1: paddr=00010360 vaddr=3f100000 size=02060h ( 16420) load-|0m
-|0:32m| (121) esp_image: segment 2: paddr=0001b454 vaddr=40080000 size=04bc4h ( 19396) load-|0m
-|0:32m| (132) esp_image: segment 3: paddr=00020020 vaddr=400d0020 size=136f0h ( 79680) map-|0m
-|0:32m| (161) esp_image: segment 4: paddr=00033718 vaddr=400840c4 size=075ech ( 30188) load-|0m
-|0:32m| (180) boot: Loaded app from partition at offset 0x10000-|0m
-|0:32m| (180) boot: Disabling RNG early entropy source...-|0m
-|0:32m| (191) cpu_start: Multicore app-|0m
-|0:32m| (192) cpu_start: Pro cpu up.-|0m
-|0:32m| (192) cpu_start: Starting app cpu, entry point is 0x40081220-|0m
-|0:32m| (0) cpu_start: App cpu up.-|0m
-|0:32m| (210) cpu_start: Pro cpu start user code-|0m
-|0:32m| (210) cpu_start: cpu freq: 168000000 Hz-|0m
-|0:32m| (210) cpu_start: Application information:-|0m
-|0:32m| (214) cpu_start: Project name: STR2024_P1_1-|0m
-|0:32m| (220) cpu_start: App version: 1-|0m
-|0:32m| (224) cpu_start: Compile time: Feb 27 2024 12:31:49-|0m
-|0:32m| (230) cpu_start: ELF file SHA256: b93922c49ce56201...-|0m
-|0:32m| (236) cpu_start: ESP-IDF: 5.1.2-|0m
-|0:32m| (241) cpu_start: Min chip rev: v0.0-|0m
-|0:32m| (246) cpu_start: Max chip rev: v3.09-|0m
-|0:32m| (251) cpu_start: Chip rev: v3.0-|0m
-|0:32m| (256) heap_init: Initializing, RAM available for dynamic allocation:-|0m
-|0:32m| (263) heap_init: At 3FFAE000 len 00001920 (6 KiB): DRAM-|0m
-|0:32m| (269) heap_init: At 3FFC2000 len 00002000 (8 KiB): DRAM-|0m
-|0:32m| (275) heap_init: At 3FFEB400 len 00003A00 (14 KiB): D/IRAM-|0m
-|0:32m| (281) heap_init: At 3FFE4350 len 00001BC0 (11 KiB): D/IRAM-|0m
-|0:32m| (288) heap_init: At 4008C100 len 00013E50 (79 KiB): IRAM-|0m
-|0:32m| (295) spi_flash: detected chip: generic-|0m
-|0:32m| (298) spi_flash: flash id: d10-|0m
-|0:32m| (303) app_start: Starting scheduler on CPU0-|0m
-|0:32m| (308) app_start: Starting scheduler on CPU1-|0m
-|0:32m| (360) main_task: Started on CPU0-|0m
-|0:32m| (318) main_task: calling app_main()-|0m
-|0:32m| (318) P1: Hello ESP32-|0m
-|0:32m| (318) P1: <Nombre>, <Apellidos>-|0m
-|0:32m| (328) P1: <Nombre>, <Apellidos>-|0m
-|0:32m| (328) P1: Modelo: ESP32, Cores: 2, Frecuencia: 168000000-|0m
-|0:32m| (338) main_task: Returned from app_main()-|0m

```



*Observa que al iniciar el monitor, el sistema se reinicia. También puedes reiniciarlo manualmente las veces que quieras pulsando el botón marcado como EN en la placa. ¡Prueba!*

## Apartado II: Primera ejecución.

Responde a las siguientes preguntas y respáldalas con capturas de pantalla:

- 1.- Haz una captura de pantalla de la salida por monitor que te da la aplicación.
- 2.- ¿Qué tipo de información te ofrece el log de inicio del sistema?
- 3.- Busca en internet la documentación del módulo `esp_chip_info.h`. Vas a ver que `Esp-idf` tiene una página de documentación clara y accesible (asegúrate de seleccionar la versión de `espidf` que está instalada en tu entorno). ¿Qué otra información podríamos extraer del sistema con este módulo?
- 4.- Modifica el código `main.c` para mostrar alguna otra información del chip a tu elección. Explica cómo lo has hecho y aporta captura de pantalla.